

Combinatorial expression cassette library for carotenoid biosynthesis in yeast

The Yeast Toolkit (YTK) and MoClo-yeast workflows use BsmBI (recognition sequence **CGTCTC**) as their Golden Gate enzyme: each cassette is flanked by outward-facing BsmBI sites that expose 4-nt overhangs for modular ligation. If an internal BsmBI site exists anywhere in the assembled construct, the enzyme will cut it during the Golden Gate reaction and shatter the cassette, hence it's important to use parts that have been 'domesticated' so they are free from restriction sites.

One issue that is harder to prevent is the junction-emergent site: neither part carries BsmBI on its own, but when two specific parts are placed in sequence the recognition site appears at their boundary. One way to prevent this is by generating all possible combinations to screen one at a time, which can quickly go exponential. We do not have to do that, however, and can use `search()` directly on the sequence graph and find matches across any combination of junctions.

Scenario: Expressing the bacterial geranylgeranyl pyrophosphate (GGPP) synthase crtE, the committed step in carotenoid biosynthesis, in *Saccharomyces cerevisiae* as part of a lycopene production strain. Three constitutive promoters and two terminators drawn from the YTK part collection are combined with two crtE variants (native and codon-optimised) in a $3 \times 2 \times 2$ factorial design of 12 expression cassettes. During cloning, you notice that some combinations appear to assemble much more efficiently than others.

Setup

```
library(genr)
library(Biostrings)

ws <- file.path(tempdir(), "yeast_carotenoid")
dir.create(ws, showWarnings = FALSE)
setwd(ws)

repo <- Repository(NULL)
```

Define parts

Parts are `DNASTringSet` objects, one column per slot. Gen stores each part's name as its annotation label; when plotting we map that name to a role (promoter, cds, terminator, or scar) and colour by role.

Backbone scars are single-part columns so every path through the graph is a complete, self-contained expression cassette, ready for Golden Gate ligation into the pYTK001 destination vector.

```
# Helper: build a named DNASTringSet for one library column.
make_parts <- function(nms, seqs) {
  DNASTringSet(setNames(seqs, nms))
}
```

```

# Upstream backbone scar, the left BsmBI assembly site.
left_scar <- make_parts(
  "left_scar",
  "AACTAGTATGGCGCGCCAACAAGGATCTGCGG"
)

# Three constitutive promoters from the YTK collection.
# TEF1p (strong) ends with ...GCGTCT, the first six bases of the BsmBI
# recognition sequence. When followed by a CDS that begins with C, the
# junction spells CGTCTC and the cassette is destroyed during assembly.
promoters <- make_parts(
  c("TEF1p", "PGK1p", "HXT7p"),
  c(
    # TEF1p: strong, ~150 bp abbreviated. Ends with GCGTCT.
    paste0(
      "AGATCTGTTGACAAATAATCATCCGGCTCGTAATGTGTGGAATTGTGAGCGGATAACAAT",
      "TTCACCAGTTCCCAACCTTACTCGGCACAAATCACCTTAATTTGATATTTATATCAAAGA",
      "ATTTCTTTTGTATGGCGTCT"
    ),
    # PGK1p: medium strength constitutive promoter.
    paste0(
      "AGTACTTTTTTTTTTATTTTTTTTTTTTTTTGAATTTTTTTTCAATTCAATT",
      "CATCATTTTTTTTTTTTTTTTTTTTCTTCTTCTTCTTCTTTTTTTTTTATTTTTT",
      "CATTTCTTTTTTTTTTTCAATT"
    ),
    # HXT7p: strong under glucose limitation (useful for fed-batch).
    paste0(
      "ATCAGTGAGCAGTATGATGGTTATATTATATAGAGCATATATAGACATATATTATATTAA",
      "ATAGATATAATTATATAATATAAGAAATAATAATATAAGAAATAATAATATAAGAAATAAT",
      "ATCAGTGAGCAACTAGCAC"
    )
  )
)

# Two crtE variants (GGPP synthase from Erwinia uredovora).
# crtE_native: unoptimised bacterial codons, ORF starts with CATG.
# TEF1p + crtE_native junction: ...GCGTCT | CATG... = CGTCTC = BsmBI site!
# crtE_coopt: S. cerevisiae codon-optimised, ORF starts with ATGG.
# TEF1p + crtE_coopt junction: ...GCGTCT | ATGG... = CGTCTA = no BsmBI site.
cds_variants <- make_parts(
  c("crtE_native", "crtE_coopt"),
  c(
    paste0(
      "CATGGTTACTATCCAGAAAACGTTGGTGCGAACATGGGCATCGTCGAAGCTGATGACGA",
      "CGCTGCGGAATCCAGTCTCAAGCTGCAATTTCCCAAATCAGACAAGCAATGATTTCTGA",
      "AGGTATTCACCAATGCAAGAAATGA"
    ),
    paste0(
      "ATGGTTACCTCTCCAGAAAACGTTGGTGCTAACATGGGTATCGTCGAAGCTGACGACGAC",
      "GCAGCTGAATCCCAATCTCAAGCAGCGATTTCCCAAATCAGACAAGCGATGATTTCTGAA",
      "GGTATCCATCAATGTAAGAAATGA"
    )
  )
)

```

```

# Two terminators from the YTK collection.
terminators <- make_parts(
  c("CYC1t", "ADH1t"),
  c(
    paste0(
      "CATGTAATTAGTTATGTCACGCTTACATTACAGCCCTCCCCACATCCGCTCTAACC AAA",
      "AATGGCCCAAACTCGCCCATTTACGTTTATTATTGATTTTTTTTTTTTTTTTCTTT",
      "TTTTTTTTTTTTTTTTTTTTTTTT"
    ),
    paste0(
      "GCGAATTTCTTATGATTTATGATTTTTATTATTTATTATTTTTATTATCTTTTTATTATT",
      "TTATTATTTATAATTTAATATATAAATAATATATTTAAATATATATTTTTTTATTATTTTAT",
      "TATTTATAATTTAATATATAAATAAT"
    )
  )
)

# Downstream backbone scar, the right BsmBI assembly site.
right_scar <- make_parts(
  "right_scar",
  "GCGGCCGCAAGCTTAGATCTCGCGGCCATACT"
)

parts_list <- list(left_scar, promoters, cds_variants, terminators, right_scar)
n_combos <- prod(lengths(parts_list))
cat(paste(lengths(parts_list), collapse = " x "), "=", n_combos, "combinations\n")

## 1 x 3 x 2 x 2 x 1 = 12 combinations

```

Import the library

`import_library()` accepts a list of `DNAStringSet` columns and builds a compact sequence graph where parts are shared nodes. Search traverses the graph directly, screening every junction simultaneously without enumerating individual constructs.

```

repo$import_library("lycopene-cassette", parts_list)

## <pointer: 0x60000189c940>
## attr(,"class")
## [1] "SequenceGraph"

sg <- Filter(function(x) x$name() == "lycopene-cassette",
  ↪ repo$get_sequence_graphs()[[1]]
cat("Sequence graph:", sg$name(), "\n")

## Sequence graph: lycopene-cassette

cat("Sample:      ", sg$sample_name(), "\n")

## Sample:      reference

```

Visualise the library graph

The annotation group added automatically on import labels every part. Because `GenPlot` calls `auto_load_annotation_groups()` at construction time, the track appears without an explicit `add_track_group()` call.

The `colors` parameter accepts a function that receives each annotation and returns a hex color string. `Gen` stores each part's name as its annotation label, so we colour by name: `ROLE_COLORS` holds the palette and `NAME_ROLE` records which role each part name belongs to.

```
ROLE_COLORS <- c(
  scar      = "#aaaaaa",
  promoter  = "#4393c3",
  cds       = "#1a9850",
  terminator = "#e08214"
)

# Gen does not persist per-part metadata, so drive colour from the part
# name (stored as the annotation label) rather than a stored role field.
NAME_ROLE <- c(
  setNames(rep("scar",      length(left_scar)),  names(left_scar)),
  setNames(rep("promoter",  length(promoters)),  names(promoters)),
  setNames(rep("cds",       length(cds_variants)), names(cds_variants)),
  setNames(rep("terminator", length(terminators)), names(terminators)),
  setNames(rep("scar",      length(right_scar)),  names(right_scar))
)
```

```
p <- plot(sg, rows = 20L,
  colors = function(ann) ROLE_COLORS[[NAME_ROLE[[ann$name]]]])
p
```



The fork-and-rejoin topology of the 12-path graph is immediately visible: three promoter branches merge at the CDS column, split again into two CDS variants, reconverge at the terminator column, and so on.

Build the design matrix

```
design <- expand.grid(
  promoter = names(promoters),
  cds      = names(cds_variants),
  terminator = names(terminators),
  stringsAsFactors = FALSE
)
design$construct_id <- seq_len(nrow(design))
design
```

```
##      promoter      cds terminator construct_id
## 1      TEF1p crtE_native      CYC1t          1
## 2      PGK1p crtE_native      CYC1t          2
## 3      HXT7p crtE_native      CYC1t          3
## 4      TEF1p crtE_coopt      CYC1t          4
## 5      PGK1p crtE_coopt      CYC1t          5
## 6      HXT7p crtE_coopt      CYC1t          6
## 7      TEF1p crtE_native      ADH1t          7
## 8      PGK1p crtE_native      ADH1t          8
## 9      HXT7p crtE_native      ADH1t          9
## 10     TEF1p crtE_coopt      ADH1t         10
## 11     PGK1p crtE_coopt      ADH1t         11
## 12     HXT7p crtE_coopt      ADH1t         12
```

Screen the full library for BsmBI

A single `search()` traverses all 12 paths simultaneously, including every inter-part junction. BsmBI (CGTCTC) would cut the cassette during the Golden Gate reaction; any internal site must be eliminated before synthesis.

```
hits <- repo$search("CGTCTC", c(sg$id()), "dna")
n_sites <- length(hits[[1]]$matches)
cat("BsmBI sites found in the library graph:", n_sites, "\n")
```

```
## BsmBI sites found in the library graph: 1
```

The search returns unique site occurrences in the sequence graph, one per distinct location in the shared graph structure. That one site sits at the TEF1p→crtE_native junction and will be traversed by every construct that pairs those two parts, regardless of which terminator follows. Translating from graph site to blocked constructs happens in the design matrix below.

Diagnose: the site is junction-emergent

One site found, yet neither part contains BsmBI individually:

```
count_bsmbi <- function(seq) {
  site <- "CGTCTC"
  n <- 0L
  for (i in seq_len(nchar(seq) - nchar(site) + 1L))
```

```

    if (substr(seq, i, i + nchar(site) - 1L) == site) n <- n + 1L
  }
}

```

```

all_parts <- c(promoters, cds_variants, terminators)
cat("BsmBI in individual parts:\n")

```

```
## BsmBI in individual parts:
```

```

for (nm in names(all_parts)) {
  n <- count_bsmbi(as.character(all_parts[[nm]]))
  flag <- if (n > 0L) " ← !" else ""
  cat(sprintf(" %-18s %d site(s)%s\n", nm, n, flag))
}

```

```

##   TEF1p           0 site(s)
##   PGK1p           0 site(s)
##   HXT7p           0 site(s)
##   crtE_native     0 site(s)
##   crtE_coopt      0 site(s)
##   CYC1t           0 site(s)
##   ADH1t           0 site(s)

```

```

tef1p_seq <- as.character(promoters[["TEF1p"]])
native_seq <- as.character(cds_variants[["crtE_native"]])
coopt_seq <- as.character(cds_variants[["crtE_coopt"]])

cat("\nTEF1p ends:           ...", substr(tef1p_seq, nchar(tef1p_seq) - 11L,
  ↪  nchar(tef1p_seq)), "\n")

```

```

##
## TEF1p ends:           ... TGTATGGCGTCT

```

```
cat("crtE_native starts:           ", substr(native_seq, 1L, 12L), "... \n")
```

```
## crtE_native starts:           CATGGTTACTAT ...
```

```
cat("crtE_coopt starts:           ", substr(coopt_seq, 1L, 12L), "... \n")
```

```
## crtE_coopt starts:           ATGGTTACCTCT ...
```

```

junction_bad <- paste0(tef1p_seq, native_seq)
junction_good <- paste0(tef1p_seq, coopt_seq)
cat("\nBsmBI at TEF1p + crtE_native junction:", count_bsmbi(junction_bad), "\n")

```

```

##
## BsmBI at TEF1p + crtE_native junction: 1

```

```
cat("BsmBI at TEF1p + crtE_coopt junction: ", count_bsmbi(junction_good), "\n")
```

```
## BsmBI at TEF1p + crtE_coopt junction: 0
```

The recognition sequence CGTCTC is split across the junction:

- TEF1p ends ...GCGTCT (the first six bases of the BsmBI octamer)
- crtE_native begins CATG... (supplying the terminal C)

Together they complete CGTCTC. This site exists on every path through the graph that pairs TEF1p with crtE_native, regardless of which terminator is chosen.

Highlight the BsmBI site on the graph

```
p2 <- plot(sg, rows=15L, cols = 60L,
           colors = function(ann) ROLE_COLORS[[NAME_ROLE[[ann$name]]]])

m <- hits[[1]]$matches[[1]]
p2$go_to_match(m, center = TRUE)
p2$highlight_match(m, color = "red")
p2
```



The highlighted region straddles the TEF1p→crtE_native edge. Because the site is stored once in the shared graph structure, a single highlight marks all paths that would be cut: here, both the CYC1t and the ADH1t variants of the TEF1p×crtE_native cassette.

Flag blocked constructs

```

design$blocked <- design$promoter == "TEF1p" & design$cds == "crtE_native"

blocked <- design[design$blocked, ]
clean <- design[!design$blocked, ]

cat(nrow(blocked), "blocked construct(s):\n")

## 2 blocked construct(s):

print(blocked[, c("construct_id", "promoter", "cds", "terminator")], row.names = FALSE)

## construct_id promoter      cds terminator
##           1   TEF1p crtE_native    CYC1t
##           7   TEF1p crtE_native    ADH1t

cat("\n", nrow(clean), "clean constructs proceed to synthesis.\n")

##
## 10 clean constructs proceed to synthesis.

```

The two blocked cassettes (TEF1p × crtE_native, regardless of terminator) cannot be assembled by Golden Gate. The remaining ten are safe.

Next steps

In practice, each part would be flanked by inward-facing BsmBI recognition sequences plus enough additional handle sequence to seat PCR primers and give the enzyme room to cut cleanly. The library graph holds the internal part content; the flanking architecture is applied at the cloning design stage.

Gen records every import and update as an operation. To attach the screening conclusion to the `import_library` operation, open the operations browser from the terminal:

```
gen operations --interactive
```

Select the lycopene-cassette import entry to add a message, for example: “Screening complete: TEF1p×crtE_native blocked by junction-emergent BsmBI site at promoter-CDS boundary. Ten combinations cleared.”

To redesign the library with a different set of parts, create a new branch from the operation just before the import. `HEAD~1` is shorthand for “one step back from the current operation” (analogous to `HEAD~1` in git):

```
gen checkout -b lycopene-v2 HEAD~1
```

Then modify the parts list in R and call `import_library()` again on the new branch. The original branch and all its screening history remain intact; you can pick up that experiment again at any point. ““